Next.js 14 App Router · TypeScript, strict · Prisma 5 · Neon Postgres · Vercel

One application, eight honest layers

A single Next.js codebase — 31 pages, 40 API routes, 30 data models. The boundary that matters most is the one between the AI layer and the verification layer, and it is placed deliberately: verification runs *after* the model and *before* anything is persisted or shown.

SYSTEM ARCHITECTURE

One Next.js application, eight honest layers

Every box below exists in the repository. Arrows show request direction; the verification layer is deliberately placed *after* the model and before persistence.

LAYER 01

Client

Mobile-first React UI, 31 pages under the App Router.

Learning screens

Scan, Solve, Learn, Gaps, Practice, Teach-back, Transfer, Exam, Focus.

`src/app/(main)/…`

Onboarding

Splash, register, login, personalize.

`src/app/(onboarding)/…`

Design system

Tailwind tokens resolved through CSS variables; themeable accent + text scale.

`tailwind.config.ts · theme.css`

LAYER 02

Edge middleware

The outer gate, running before any route.

Session verification

JWT signature actually verified at the edge with `jose` — not a "cookie exists" check.

`src/middleware.ts`

Route gating

Public pages and public APIs are allow-listed; everything else redirects to /splash.

OAuth exemption

Spotify callbacks bypass the redirect so the handler can spend its single-use PKCE cookies and explain the outcome.

LAYER 03

API routes

40 route handlers. Each re-checks the session itself.

Diagnosis

analyses · uploads · solve · explain

Repair & mastery

gaps · practice-attempts · teach-back · transfer-attempts · exam

State

roadmap · history · knowledge · settings · achievements · reports

Integrations

resources · concept-image · spotify/*

LAYER 04

AI orchestration

Interpretation only. Never the final authority.

Provider cascade

cache → Gemini → Groq → caller's deterministic fallback. Every attempt logged with the provider that made it.

`lib/ai/ai-client.ts`

Pipeline stages

analyze-work · reconstruct-reasoning · classify-gap · explain-gap · generate-practice · evaluate-teachback

`lib/ai/pipeline/`

RAG retrieval

Local TF-IDF-style keyword scoring over 58 curated chunks. No vector database.

`lib/ai/rag/retrieve.ts`

LAYER 05

Verification

The layer that decides what is *true*. Zero AI.

Algebra verifier

mathjs proportionality check: does the transformation preserve the solution set?

`verification/math-verifier.ts`

Domain routing

Per-step-pair: algebra, quantitative, or chemical — chosen from the shape of the lines, not the subject.

`verification/verify-step.ts`

Misconception catalogue

17 codes plus UNCLASSIFIED. Where the signature is unambiguous the code is proved outright — no model involved.

`diagnosis/misconceptions.ts`

"Uncertain" is a result

Unparseable lines are never reported as errors. Silence beats a false accusation.

APPLICATION FLOW

One request, traced end to end

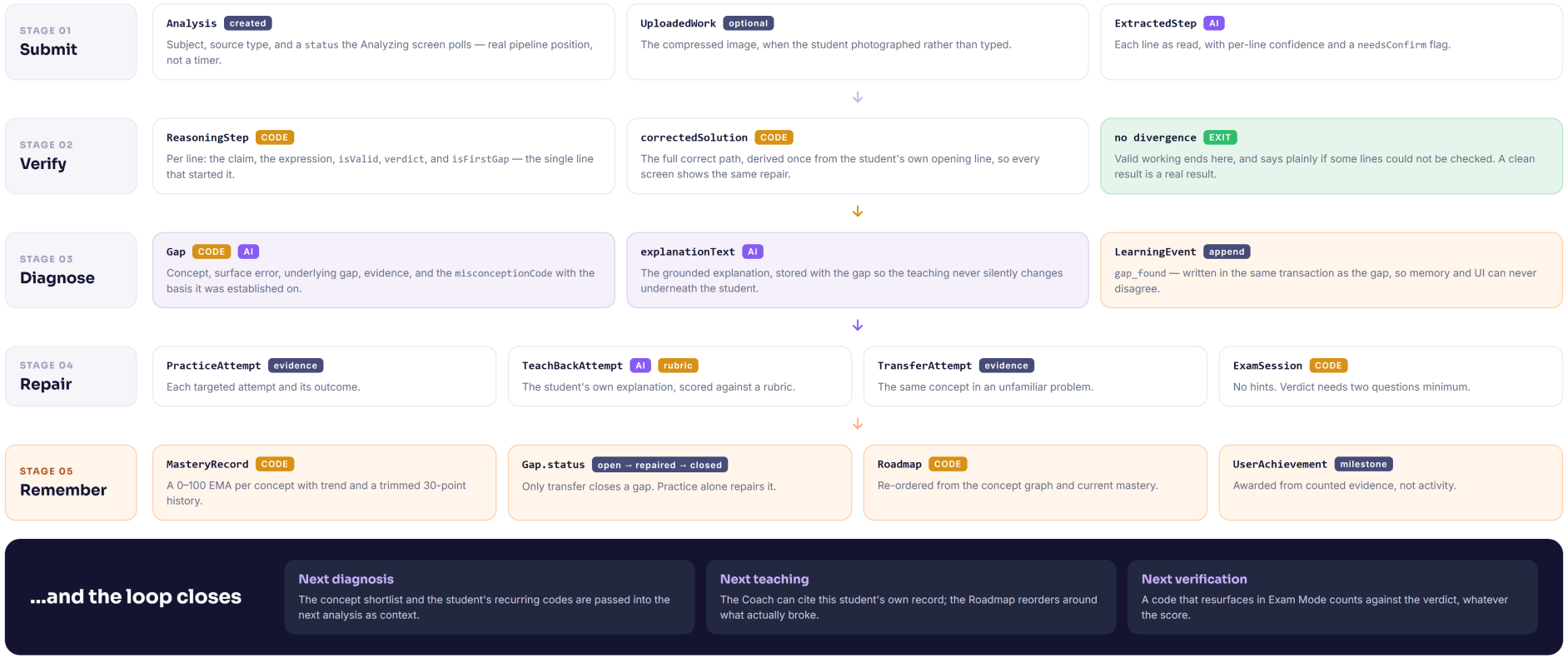
`POST /api/analyses` returns in roughly a quarter of a second with an analysis id, long before the pipeline finishes. The screen the student then watches is polling real pipeline position — a status the orchestrator writes as it enters each stage.

STEP 01	**Client**	**Capture and compress** The photograph is downscaled and compressed *in the browser*, so a large phone image crosses the network as a fraction of its bytes. Alternatively the student types their lines, which skips the vision stage entirely.
STEP 02	**Edge middleware**	**Verify the session before the route runs** `src/middleware.ts` verifies the JWT signature at the edge with `jose` — not a "does a cookie exist" check. Public pages and public APIs are allow-listed; everything else redirects to `/splash`.
STEP 03	**API route**	**Re-check, create, return immediately** The handler calls `getSessionUserId()` itself — middleware is the outer gate, not the only one. It creates the `Analysis` row with `status: processing` and returns the id. The pipeline continues after the response.
STEP 04	**AI orchestration**	**One multimodal call, through the cascade** `cache → Gemini → Groq → deterministic`. Reading, narrating and classifying are collapsed into a single request whose output is carried forward. Low confidence pauses at `needs_confirmation` and asks the student instead of guessing.
STEP 05	**Verification**	**The gate — zero AI beyond this point in the decision** Each adjacent pair of lines is routed *by shape, not by subject* — a chemistry paper is full of algebra, and that algebra is still checkable. `mathjs` tests whether the transformation preserves the solution set; the audit then locates the first divergence, classifies every later line, and derives the corrected line.
STEP 06	**Diagnosis**	**Prove the code where the signature allows it** Signature detection runs first: where an algebraic fingerprint identifies the error outright the catalogue code is **proved**. Otherwise the model's pick is validated against the catalogue and labelled **matched**. An unrecognised code becomes `UNCLASSIFIED` rather than being invented.
STEP 07 · 08	**Persistence & observability**	**Write once at the moment of computation — and log the call, not just the result** Every artefact — extracted steps, reasoning steps, the gap, the corrected solution, the explanation — is persisted as it is computed, so reopening a page never regenerates anything and never changes what the student already read. Every provider attempt also writes an `AiUsageLog` row, which is what makes the trace view possible.

DATA FLOW

What one attempt leaves behind

Each stage persists its own structured artefact, so the system can always answer "what did it actually do here?" — and so the next attempt starts informed.



GapFinder

Diagram 05 — Data flow

DATA MODEL

30 models, and the one join that matters

`Gap.misconceptionCode` is what makes longitudinal tracking possible: the same code appearing in a homework diagnosis and in an exam answer is recognised as the same event, because both came from the same catalogue.

User

Owns everything. Every query in the application is scoped by `userId`. Profile, StudyPreference, UserSettings and SpotifyAccount hang off it.

Analysis

One submitted attempt. Carries subject, source type and the `status` the Analyzing screen polls.

ExtractedStep

AI Each line as read, with per-line confidence and a `needsConfirm` flag.

ReasoningStep

CODE Per line: the claim, the expression, `isValid`, `verdict`, and `isFirstGap`.

Gap

The diagnosis. Concept, surface error, underlying gap, evidence, confidence, explanation, `misconceptionCode`, the `basis` it was established on, and `status: open | repaired | closed`.

Concept

22 seeded, across four subjects. Joined to KnowledgeChunk, ExamQuestion and ConceptRelationship (29 prerequisite edges).

KnowledgeChunk

58 curated entries, typed by kind — explanation, worked example, misconception, teaching strategy, practice pattern.

MasteryRecord

CODE One 0–100 EMA per user per concept, with trend and a trimmed 30-point history.

PracticeAttempt

Targeted repair attempts and their outcomes, joined to the Gap they belong to.

TransferAttempt

The same concept in an unfamiliar problem. The only event that moves a gap to `closed`.

TeachBackAttempt

The student's own explanation, scored against a rubric which then sets the mastery target directly.

ExamSession
ExamQuestion

No hints, no feedback. Per-concept verdicts requiring at least two answered questions.

LearningEvent

Append-only. `gap_found`, `transfer_success`, `teach_back`, `mastery_change` — written alongside the record they describe.

LearningMemory

One row per user: recurring gaps with counts, successful and failed repairs, transfer results. Each list trimmed to the last 50.

Roadmap
Recommendation

The next-action ordering, built from the concept graph against current mastery.

AiCallCache
AiUsageLog

Content-hashed responses with a 7-day TTL, and one log row per model call — the observability substrate.

Two deliberate asymmetries. `ExtractedStep` and `ReasoningStep` are separate tables because the first is what a model *read* and the second is what code *verified*. And `Gap.misconceptionBasis` exists so that "proved" and "probably" are stored — and shown — differently.

TECH STACK

Every technology, and the job it does

Only technologies actually present in `package.json` and used in the running application.

TECHNOLOGY	ROLE	WHY
Next.js 14 App Router	Framework	One deployment for UI and API; server components keep the learner record off the client
TypeScript strict	Language	`noUncheckedIndexedAccess` — a step array indexed past its end is a compile error, not a runtime accusation
mathjs	Verification	The layer that decides truth. Symbolic evaluation, not string comparison
Prisma 5	ORM	Typed access across 30 models; the schema is the single source of truth
Neon Postgres	Database	Serverless Postgres reached over its WebSocket driver on 443
Google Gemini	Vision + reasoning	The strongest multimodal reading available at this tier, with structured output
Groq	Text fallback	A *separate* free tier, fast enough that failing over costs the student almost nothing

TECHNOLOGY	ROLE	WHY
Zod + `zod-to-json-schema`	Schema	One definition drives both validation and the providers' structured-output contracts
jose	Auth	Edge-compatible, so the JWT signature is verified in middleware rather than trusted
Tailwind + CSS variables	Styling	Themeable accent and text scale without a second styling system
Framer Motion **Recharts**	UI	The reasoning replay, and mastery trends over time
sharp · **bcryptjs**	Media · auth	Server-side image preparation; password hashing that never leaves the server
Vitest	Tests	289 tests across 18 files, all runnable with no API keys
Vercel	Hosting	Framework preset; AI-bearing routes declare their own `maxDuration`

ENGINEERING DECISIONS

Five choices, and one bug that shaped the rest

Each of these is a place where the easy implementation would have been to ask a language model, and the system deliberately does something else.

WHY SEPARATE AI FROM VERIFICATION AT ALL?

Because a false accusation is unrecoverable

A student wrongly told their correct line is wrong stops trusting every correct thing the product says afterwards. An LLM asked "is this step right?" is confidently wrong often enough to make that a certainty at scale. So correctness is proved, and the model is confined to reading and phrasing.

WHY TWO AI PROVIDERS?

Because one free tier is a single point of failure

Groq's quota is independent of Google's, so exhausting Gemini does not stop the product. A `quota` error is never retried — it is not transient — and a malformed request is never failed over, because the second provider would reject it too.

WHY NEON OVER WEBSOCKETS?

Because campus wifi blocks 5432

Direct Postgres sockets are blocked on many of the networks students actually study on, which made every database-backed page fail with an unhelpful error. Port 443 is never blocked, and it is the same transport the serverless runtime uses in production.

WHY NO VECTOR DATABASE?

Because 58 hand-written chunks do not need one

Local keyword scoring over a curated corpus is free, has no network hop, and is auditable — a human can read the whole corpus. An embedding service would add cost and latency to buy relevance the corpus size does not require.

WHY PERSIST EVERY ARTEFACT?

Because teaching must not change underneath a learner

Every artefact is written at the moment it is computed and read back thereafter, so reopening an analysis never re-runs it. That protects both trust — last week's diagnosis is still the one shown — and a free-tier quota.

THE BUG THAT SHAPED THE STRUCTURE

Middleware at the repo root is silently ignored

With a `src/` directory, Next.js does not look for `middleware.ts` at the root — and gives no warning. Every protected page was publicly reachable. It now lives at `src/middleware.ts`, and cross-user isolation is verified explicitly rather than assumed.

SECURITY, DELIVERY, OBSERVABILITY

The boundaries that are not about learning

Kept short deliberately. These are the parts that must simply be correct.

SECURITY

Route protection	`src/middleware.ts` verifies the JWT with `jose` on every protected path; each API route re-checks the session itself
Session secret	Rejected if absent or under 16 characters, with **no fallback secret** — a deploy that forgot it rejects every session rather than verifying against a published value
Cross-user access	Every query scoped by `userId`; another user's analysis returns **404, not 403** — a 403 would confirm the record exists
Login timing	Constant-time comparison; an unknown user and a wrong password produce the identical response
Passwords	`bcryptjs` hashes; the hash never leaves the server
Uploads	Strict MIME allowlist, size ceiling, client-side downscale before transmission
Secrets	Environment variables only — never logged, returned to a client, or committed. `.env.example` documents each one
OAuth	Spotify runs server-side with PKCE and single-use cookies; the callback is exempt from the redirect gate *only* so the handler can check the session itself

DEPLOYMENT

Platform	Vercel, Next.js framework preset (`vercel.json`)
Functions	AI-bearing routes declare `maxDuration` of 30–60s; the rest use the default
Post-response work	`waitUntil` lets the response return before background persistence finishes
Schema changes	Applied over HTTPS where 5432 is blocked (`npm run db:apply`)
Verification gate	`npm run verify` — lint, typecheck, tests, eval. Passes with **no API keys at all**

OBSERVABILITY

Per call	Stage, `provider:model`, cached, succeeded, latency, error text, retrieved chunk ids
Per analysis	A trace view replaying which stage ran, who answered it, and why the first choice did not
What it makes visible	Failover (Gemini declined, Groq answered) and fallback (deterministic logic produced the result) — normally invisible events